

An Introduction to Process Calculi

Gabriele Cecilia

University of Milan
Dept. of Mathematics

13 December 2023

Table of Contents

1 Introduction

2 CCS

- Syntax
- Semantics
- Behavioural Equivalence

3 Value Passing CCS

- Syntax
- Semantics

4 π -calculus

- Syntax
- Semantics
 - Labelled Transition Semantics
 - Reduction Semantics
- Behavioural Equivalence

Concurrent systems are systems (programs, problems, environments, ...) in which different units or processes can perform operations independently or simultaneously with each other.

Examples: concurrent programming, railway systems, ...

Problems: deadlock, correctness, efficiency, ...

Process calculi (or algebras) are mathematical models which allow to formally describe concurrent systems.

They provide:

- formal specification of processes
- description of process behaviour and interactions
- analysis and verification of process properties

Table of Contents

1 Introduction

2 CCS

- Syntax
- Semantics
- Behavioural Equivalence

3 Value Passing CCS

- Syntax
- Semantics

4 π -calculus

- Syntax
- Semantics
 - Labelled Transition Semantics
 - Reduction Semantics
- Behavioural Equivalence

Calculus of Communicating Systems (R. Milner, 1980)

In CCS each process is a black box, with a name and a collection of communicating ports (*channels*) used for input/output.

Operational semantics can be given in terms of automata: processes can be represented by vertices of certain labelled oriented graphs, and a change of process state caused by performing an action can be seen as moving along a certain edge.

Calculus of Communicating Systems (R. Milner, 1980)

In CCS each process is a black box, with a name and a collection of communicating ports (*channels*) used for input/output.

Operational semantics can be given in terms of automata: processes can be represented by vertices of certain labelled oriented graphs, and a change of process state caused by performing an action can be seen as moving along a certain edge.

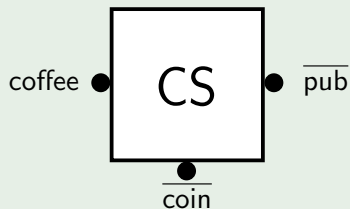
It can be used to describe simple situations in which different units interact with each other. The focus is on *interactions* themselves, rather than data exchanged.

Examples: people using vending machines, vehicles using toll booths, ...

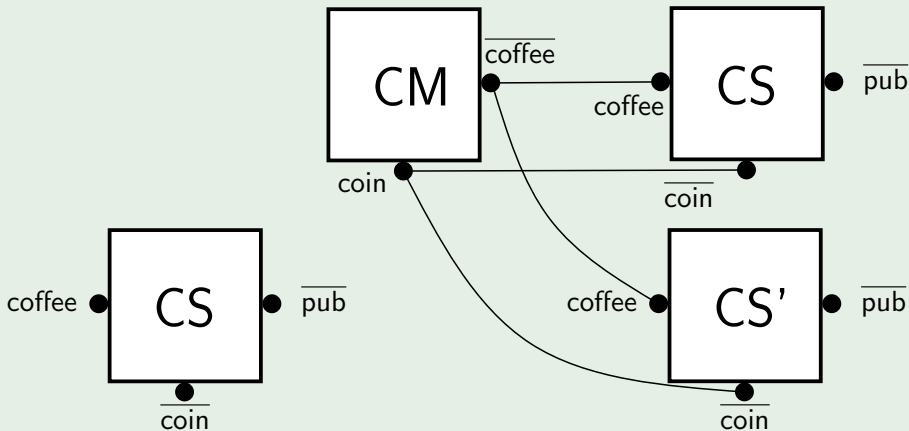
More specifically, CCS models *communication*, *parallel composition* of processes, *choice* of actions, *relabeling* and *scope restriction*; its base presentation doesn't support exchange of data between processes or scope extension.

It can be extended so that process are enabled to send or receive some sort of data (*Value Passing CCS*).

Examples of processes



Examples of processes



Let's assume the existence of a countably infinite set $\mathcal{A}$ of (channel) **names**.

We denote the set of **complementary names** as $\bar{\mathcal{A}} = \{\bar{a} \mid a \in \mathcal{A}\}$.

Let's assume the existence of a countably infinite set $\mathcal{A}$ of (channel) **names**.

We denote the set of **complementary names** as $\bar{\mathcal{A}} = \{\bar{a} \mid a \in \mathcal{A}\}$.

Definition

We define the set of **actions** as $Act = \mathcal{A} \cup \bar{\mathcal{A}} \cup \{\tau\}$.

Let's assume the existence of a countably infinite set $\mathcal{A}$ of (channel) **names**.

We denote the set of **complementary names** as $\bar{\mathcal{A}} = \{\bar{a} \mid a \in \mathcal{A}\}$.

Definition

We define the set of **actions** as $Act = \mathcal{A} \cup \bar{\mathcal{A}} \cup \{\tau\}$.

Actions given by channel names represent *input actions*.

Actions given by overlined channel names represent *output actions*.

τ is a standard label which represents *communication* between two processes changing their state simultaneously.

Definition

The syntax of **CCS processes** is the following:

$$P, Q := 0 \mid K \mid \alpha.P \mid P + Q \mid P \mid Q \mid P[f] \mid P \setminus L$$

where K is a constant process, $\alpha \in Act$, f is a function from Act to Act such that $f(\bar{a}) = \overline{f(a)}$ and $f(\tau) = \tau$, $L \subseteq \mathcal{A} \cup \bar{\mathcal{A}}$.

Definition

The syntax of **CCS processes** is the following:

$$P, Q := 0 \mid K \mid \alpha.P \mid P + Q \mid P \mid Q \mid P[f] \mid P \setminus L$$

where K is a constant process, $\alpha \in Act$, f is a function from Act to Act such that $f(\bar{a}) = \overline{f(a)}$ and $f(\tau) = \tau$, $L \subseteq \mathcal{A} \cup \bar{\mathcal{A}}$.

We denote the set of CCS processes as $Proc$.

We also allow definitions of constant process through their (eventually recursive) defining equation $K \stackrel{\text{def}}{=} P$.

Examples of processes

$$CM \stackrel{\text{def}}{=} \text{coin}.\overline{\text{coffee}}.CM$$
$$CS \stackrel{\text{def}}{=} \overline{\text{pub}}.\overline{\text{coin}}.\text{coffee}.CS$$

Examples of processes

$$CM \stackrel{\text{def}}{=} \text{coin}.\overline{\text{coffee}}.CM$$
$$CS \stackrel{\text{def}}{=} \overline{\text{pub}}.\overline{\text{coin}}.\text{coffee}.CS$$
$$CTM \stackrel{\text{def}}{=} \text{coin}.\overline{\text{coffee}}.CTM + \overline{\text{tea}}.CTM$$
$$CM \mid CS \stackrel{\text{def}}{=} (\text{coin}.\overline{\text{coffee}}.CM) \mid (\overline{\text{pub}}.\overline{\text{coin}}.\text{coffee}.CS)$$

Examples of processes

$$CM \stackrel{\text{def}}{=} \text{coin}.\overline{\text{coffee}}.CM$$

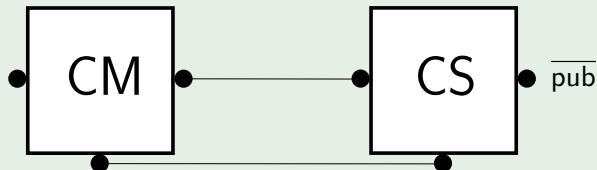
$$CS \stackrel{\text{def}}{=} \overline{\text{pub}}.\overline{\text{coin}}.\text{coffee}.CS$$

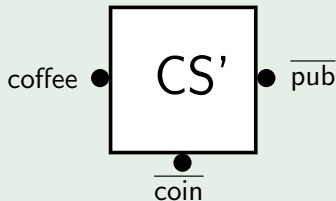
$$CTM \stackrel{\text{def}}{=} \text{coin}.\overline{(\text{coffee}.CTM + \overline{\text{tea}}.CTM)}$$

$$CM \mid CS \stackrel{\text{def}}{=} (\text{coin}.\overline{\text{coffee}}.CM) \mid (\overline{\text{pub}}.\overline{\text{coin}}.\text{coffee}.CS)$$

$$VM \stackrel{\text{def}}{=} \text{coin}.\overline{\text{item}}.VM \quad \rightarrow \quad CM \stackrel{\text{def}}{=} VM[\text{coffee}/\text{item}]$$

Examples of processes

$$SM \stackrel{\text{def}}{=} (CM \mid CS) \setminus \text{coin} \setminus \text{coffee}$$


$$SM \mid CS'$$


Process behaviour in CCS is defined through a **labelled transition semantics**: processes make transitions through actions, returning another process.

Transitions describe change of state of processes after input/output or interaction with other processes.

Process behaviour in CCS is defined through a **labelled transition semantics**: processes make transitions through actions, returning another process.

Transitions describe change of state of processes after input/output or interaction with other processes.

Definition

We define the **transition** relation $\xrightarrow{(-)}$ as the smallest relation which is a subset of $Proc \times Act \times Proc$ and satisfies the following rules:

Definition

$$\frac{\text{S-ACT}}{\alpha.P \xrightarrow{\alpha} P}$$

$$\frac{\text{S-SUM-L} \quad P \xrightarrow{\alpha} P'}{P + Q \xrightarrow{\alpha} P'}$$

$$\frac{\text{S-SUM-R} \quad Q \xrightarrow{\alpha} Q'}{P + Q \xrightarrow{\alpha} Q'}$$

$$\frac{\text{S-PAR-L} \quad P \xrightarrow{\alpha} P'}{P \mid Q \xrightarrow{\alpha} P' \mid Q}$$

$$\frac{\text{S-PAR-R} \quad Q \xrightarrow{\alpha} Q'}{P \mid Q \xrightarrow{\alpha} P \mid Q'}$$

$$\frac{\text{S-COM} \quad P \xrightarrow{a} P' \quad Q \xrightarrow{\bar{a}} Q'}{P \mid Q \xrightarrow{\tau} P' \mid Q'}$$

$$\frac{\text{S-REL} \quad P \xrightarrow{\alpha} P'}{P[f] \xrightarrow{\alpha} P'[f]}$$

$$\frac{\text{S-RES} \quad P \xrightarrow{\alpha} P' \quad \alpha, \bar{\alpha} \notin L}{P \setminus L \xrightarrow{\alpha} P' \setminus L}$$

$$\frac{\text{S-DEF} \quad P \xrightarrow{\alpha} P' \quad K \stackrel{\text{def}}{=} P}{K \xrightarrow{\alpha} P'}$$

Examples of transitions

- Behaviour of a coffee and tea machine CTM, with
 $\text{CTM} \stackrel{\text{def}}{=} \text{coin}.\overline{\text{coffee}}.\text{CTM} + \overline{\text{tea}}.\text{CTM}$

Examples of transitions

- Behaviour of a coffee and tea machine CTM, with

$$\text{CTM} \stackrel{\text{def}}{=} \text{coin}.\overline{\text{coffee}}.\text{CTM} + \overline{\text{tea}}.\text{CTM}$$

First, the machine receives a coin as an input:

$$\text{S-ACT} \frac{}{\text{coin}.\overline{\text{coffee}}.\text{CTM} + \overline{\text{tea}}.\text{CTM} \xrightarrow{\text{coin}} \overline{\text{coffee}}.\text{CTM} + \overline{\text{tea}}.\text{CTM}}$$

Examples of transitions

- Behaviour of a coffee and tea machine CTM, with

$$\text{CTM} \stackrel{\text{def}}{=} \text{coin}.\overline{\text{coffee}}.\text{CTM} + \overline{\text{tea}}.\text{CTM}$$

First, the machine receives a coin as an input:

$$\text{S-ACT} \frac{}{\text{coin}.\overline{\text{coffee}}.\text{CTM} + \overline{\text{tea}}.\text{CTM} \xrightarrow{\text{coin}} \overline{\text{coffee}}.\text{CTM} + \overline{\text{tea}}.\text{CTM}}$$

Then, the machine can either output tea or coffee, getting back to the initial configuration:

$$\text{S-SUM-R} \frac{\overline{\text{tea}}.\text{CTM} \xrightarrow{\text{tea}} \text{CTM}}{\overline{\text{coffee}}.\text{CTM} + \overline{\text{tea}}.\text{CTM} \xrightarrow{\text{tea}} \text{CTM}}$$

Examples of transitions

- Behaviour of the system $\{\text{coffee machine, computer scientist}\}$

$\text{CS} \mid \text{CM, with CM} \mid \text{CS} \stackrel{\text{def}}{=} (\text{coin}.\overline{\text{coffee}}.\text{CM}) \mid (\overline{\text{pub}}.\overline{\text{coin}}.\text{coffee}.\text{CS})$

Examples of transitions

- Behaviour of the system $\{\text{coffee machine, computer scientist}\}$
 $\text{CS} \mid \text{CM}, \text{ with } \text{CM} \mid \text{CS} \stackrel{\text{def}}{=} (\text{coin}.\overline{\text{coffee}}.\text{CM}) \mid (\overline{\text{pub}}.\overline{\text{coin}}.\text{coffee}.\text{CS})$

First, the computer scientist produces a publication:

$$\text{S-PAR-R} \frac{\text{CS} \xrightarrow{\overline{\text{pub}}} \overline{\text{coin}}.\text{coffee}.\text{CS}}{\text{CM} \mid \text{CS} \xrightarrow{\overline{\text{pub}}} \text{CM} \mid \overline{\text{coin}}.\text{coffee}.\text{CS}}$$

Examples of transitions

- Behaviour of the system {coffee machine, computer scientist}

$$\text{CS} \mid \text{CM}, \text{ with } \text{CM} \mid \text{CS} \stackrel{\text{def}}{=} (\text{coin}.\overline{\text{coffee}}.\text{CM}) \mid (\overline{\text{pub}}.\overline{\text{coin}}.\text{coffee}.\text{CS})$$

Then, the scientist gives a coin to the machine: the two processes are interacting, changing their state simultaneously.

$$\text{S-COM} \frac{\text{CM} \xrightarrow{\text{coin}} \overline{\text{coffee}}.\text{CM} \quad \overline{\text{coin}}.\text{coffee}.\text{CS} \xrightarrow{\overline{\text{coin}}} \text{coffee}.\text{CS}}{\text{CM} \mid \overline{\text{coin}}.\text{coffee}.\text{CS} \xrightarrow{\tau} \overline{\text{coffee}}.\text{CM} \mid \text{coffee}.\text{CS}}$$

Examples of transitions

- Behaviour of the system $\{\text{coffee machine, computer scientist}\}$
 $\text{CS} \mid \text{CM}$, with $\text{CM} \mid \text{CS} \stackrel{\text{def}}{=} (\text{coin}.\overline{\text{coffee}}.\text{CM}) \mid (\overline{\text{pub}}.\overline{\text{coin}}.\text{coffee}.\text{CS})$

Finally, the machine gives coffee to the scientist.

$$\text{S-COM} \frac{\overline{\text{coffee}}.\text{CM} \xrightarrow{\text{coffee}} \text{CM} \quad \text{coffee}.\text{CS} \xrightarrow{\text{coffee}} \text{CS}}{\overline{\text{coffee}}.\text{CM} \mid \text{coffee}.\text{CS} \xrightarrow{\tau} \text{CM} \mid \text{CS}}$$

When can we say that two processes are **equivalent**?

When can we say that two processes are **equivalent**?

Two processes are equivalent when they have the *same behaviour*, i.e. when the same *observations* about them can be made.

Equivalent process then have the same properties, such as the possibility of deadlock or making a certain transition.

According to different interpretations of *behaviour* or *observations*, there are different notions of behavioural equivalence.

Strong bisimilarity requires that equivalent processes can make transitions through the same actions, reaching processes which, in turn, are equivalent.

According to different interpretations of *behaviour* or *observations*, there are different notions of behavioural equivalence.

Strong bisimilarity requires that equivalent processes can make transitions through the same actions, reaching processes which, in turn, are equivalent.

According to strong bisimilarity, two processes P and $\tau.P$ are not equivalent; though, we might say that the τ -labelled transition is *internal* and thus *unobservable*.

Weak bisimilarity is a notion of bisimilarity that allows to abstract from internal transitions in process behaviours.

Definition

A binary relation $\mathcal{R}$ over $Proc$ is a **strong bisimulation** if, whenever $P \mathcal{R} Q$ and α is an action:

- if $P \xrightarrow{\alpha} P'$, then there is a process Q' such that $Q \xrightarrow{\alpha} Q'$ and $P' \mathcal{R} Q'$;
- if $Q \xrightarrow{\alpha} Q'$, then there is a process P' such that $P \xrightarrow{\alpha} P'$ and $P' \mathcal{R} Q'$.

Definition

A binary relation $\mathcal{R}$ over $Proc$ is a **strong bisimulation** if, whenever $P \mathcal{R} Q$ and α is an action:

- if $P \xrightarrow{\alpha} P'$, then there is a process Q' such that $Q \xrightarrow{\alpha} Q'$ and $P' \mathcal{R} Q'$;
- if $Q \xrightarrow{\alpha} Q'$, then there is a process P' such that $P \xrightarrow{\alpha} P'$ and $P' \mathcal{R} Q'$.

Definition

Two processes P and Q are **strongly bisimilar** ($P \sim Q$) if there is a strong bisimulation that relates them.

It can be shown that strong bisimilarity $\sim$ is *the largest strong bisimulation*: it is a bisimulation itself and it contains every other bisimulation.

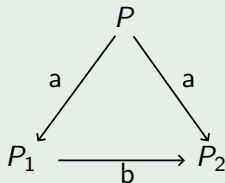
It follows that strong bisimulation is an example of **coinductive predicate**: it is the largest relation satisfying certain conditions.

It can be shown that strong bisimilarity $\sim$ is *the largest strong bisimulation*: it is a bisimulation itself and it contains every other bisimulation.

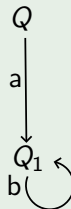
It follows that strong bisimulation is an example of **coinductive predicate**: it is the largest relation satisfying certain conditions.

On the other hand, the other relations defined in this presentation (such as the transition relation) are *inductive predicates*.

Example of bisimulation

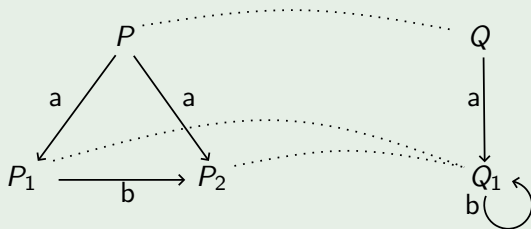


$$\begin{aligned}
 P &\stackrel{\text{def}}{=} a.P_1 + a.P_2 \\
 P_1 &\stackrel{\text{def}}{=} b.P_2 \\
 P_2 &\stackrel{\text{def}}{=} b.P_1
 \end{aligned}$$



$$\begin{aligned}
 Q &\stackrel{\text{def}}{=} a.Q_1 \\
 Q_1 &\stackrel{\text{def}}{=} b.Q_1
 \end{aligned}$$

Example of bisimulation



$$\mathcal{R} = \{(P, Q), (P_1, Q_1), (P_2, Q_1)\}$$

Table of Contents

1 Introduction

2 CCS

- Syntax
- Semantics
- Behavioural Equivalence

3 Value Passing CCS

- Syntax
- Semantics

4 π -calculus

- Syntax
- Semantics
 - Labelled Transition Semantics
 - Reduction Semantics
- Behavioural Equivalence

Value Passing CCS

Syntax

In Value Passing CCS, processes can also **send and receive data** (e.g. natural numbers).

Value Passing CCS

Syntax

In Value Passing CCS, processes can also **send and receive data** (e.g. natural numbers).

Definition

We define the set of **actions** (Act) as follows:

$$\alpha, \beta := a(n) \mid \bar{a}(n) \mid \tau \quad \text{where } a \in Names, n \in \mathbb{N}.$$

Value Passing CCS

Syntax

Definition

We define the set of **processes** (*Proc*) as follows:

$$P, Q := 0 \mid K \mid a(x).P \mid \bar{a}(e).P \mid \tau.P \mid P+Q \mid P \mid Q \mid P[f] \mid P \setminus L$$

where K is a constant process, $a \in \text{Names}$, x is a variable, e is an expression in $\mathbb{N}$, f is a function from Act to Act such that $f(\bar{a}) = \overline{f(a)}$ and $f(\tau) = \tau$, $L \subseteq \mathcal{A} \cup \bar{\mathcal{A}}$.

Equations defining constant processes can now carry a parameter, such as in $K(x) \stackrel{\text{def}}{=} P$.

Definition

We define the **transition** relation $\xrightarrow{(-)}$ as the smallest relation which satisfies the same CCS rules, but with the following new operational rules for **input/output prefixing** and for **parametrized constant processes**:

$$\text{S-IN} \frac{}{a(x).P \xrightarrow{a(n)} P\{n/x\}} \quad n \in \mathbb{N}$$

$$\text{S-OUT} \frac{}{\bar{a}(e).P \xrightarrow{\bar{a}(n)} P} \quad e \text{ has value } n$$

$$\text{S-DEF} \frac{P\{n/x\} \xrightarrow{\alpha} P' \quad K(x) \stackrel{\text{def}}{=} P}{K(e) \xrightarrow{\alpha} P'} \quad e \text{ has value } n$$

Examples of processes and transitions

In Value Passing CCS we can define a simple calculator Calc , which receives a natural number as input and outputs (for example) its successor.

$$\text{Calc} \stackrel{\text{def}}{=} \text{in}(x).\text{Calc}(x)$$

$$\text{Calc}(x) \stackrel{\text{def}}{=} \overline{\text{out}}(x + 1).\text{Calc}$$

Examples of processes and transitions

In Value Passing CCS we can define a simple calculator Calc , which receives a natural number as input and outputs (for example) its successor.

$$\text{Calc} \stackrel{\text{def}}{=} \text{in}(x).\text{Calc}(x)$$

$$\text{Calc}(x) \stackrel{\text{def}}{=} \overline{\text{out}}(x + 1).\text{Calc}$$

The calculator receives the input:

$$\text{S-IN} \frac{}{\text{in}(x).\text{Calc}(x) \xrightarrow{\text{in}(n)} \text{Calc}(n)}$$

Examples of processes and transitions

In Value Passing CCS we can define a simple calculator Calc, which receives a natural number as input and outputs (for example) its successor.

$$\text{Calc} \stackrel{\text{def}}{=} \text{in}(x).\text{Calc}(x)$$

$$\text{Calc}(x) \stackrel{\text{def}}{=} \overline{\text{out}}(x + 1).\text{Calc}$$

Then, the result of the expression " $x + 1$ " with $n = x$ is evaluated and the calculator outputs the result:

$$\text{S-OUT} \frac{}{\overline{\text{out}}(n + 1).\text{Calc} \xrightarrow{\overline{\text{out}}(n+1)} \text{Calc}}$$

Table of Contents

1 Introduction

2 CCS

- Syntax
- Semantics
- Behavioural Equivalence

3 Value Passing CCS

- Syntax
- Semantics

4 π -calculus

- Syntax
- Semantics
 - Labelled Transition Semantics
 - Reduction Semantics
- Behavioural Equivalence

In π -calculus there is no difference between channel names and data exchanged between processes: **channel names are transferred along the channels themselves.**

In π -calculus there is no difference between channel names and data exchanged between processes: **channel names are transferred along the channels themselves.**

Therefore, π -calculus models processes whose interconnections change as they interact.

Let's assume the existence of an infinite countable set of **names**, represented by the symbols $x, y, \dots$.

Definition

The syntax of **processes** is the following:

$$P, Q := 0 \mid x(y).P \mid \bar{x}y.P \mid P + Q \mid P \mid Q \mid !P \mid (\nu x)P$$

Let's assume the existence of an infinite countable set of **names**, represented by the symbols $x, y, \dots$.

Definition

The syntax of **processes** is the following:

$$P, Q := 0 \mid x(y).P \mid \bar{x}y.P \mid P + Q \mid P \mid Q \mid !P \mid (\nu x)P$$

Recursive definitions through defining equations have been replaced by replication, and only one name is restricted at a time.

Other process constructs can be added to π -calculus, such as the match operator (*if $x = y$ then P*) or mismatch operator (*if $x \neq y$ then P*).

Semantics can be given either through **labelled transitions** as before, or through **congruence** and **reduction**.

Definition

We define the set of **actions** (Act) as follows:

$$\alpha, \beta := x(y) \mid \bar{x}y \mid \bar{x}(y) \mid \tau$$

The action $\bar{x}(y)$ represents a bound output action: the bound name y is sent along x .

Definition

We define the set of **actions** (Act) as follows:

$$\alpha, \beta := x(y) \mid \bar{x}y \mid \bar{x}(y) \mid \tau$$

The action $\bar{x}(y)$ represents a bound output action: the bound name y is sent along x .

Definition

Given an action α , we define $n(\alpha)$, $fn(\alpha)$ and $bn(\alpha)$ as the **names occurring in α** (free/bound respectively).

A similar definition for names occurring in processes can be given.

Definition

We define the **transition relation** $\xrightarrow{(-)}$ as the smallest relation which is a subset of $Proc \times Act \times Proc$ and satisfies the following rules:

$$\text{S-IN} \frac{}{x(z).P \xrightarrow{x(y)} P\{y/z\}}$$

$$\text{S-OUT} \frac{}{\bar{x}y.P \xrightarrow{\bar{x}y} P}$$

$$\text{S-TAU} \frac{}{\tau.P \xrightarrow{\tau} P}$$

$$\text{S-SUM-L} \frac{P \xrightarrow{\alpha} P'}{P + Q \xrightarrow{\alpha} P'}$$

$$\text{S-PAR-L} \frac{P \xrightarrow{\alpha} P' \quad \text{bn}(\alpha) \cap \text{fn}(Q) = \emptyset}{P \mid Q \xrightarrow{\alpha} P' \mid Q}$$

$$\text{S-SUM-R} \frac{Q \xrightarrow{\alpha} Q'}{P + Q \xrightarrow{\alpha} Q'}$$

$$\text{S-PAR-R} \frac{Q \xrightarrow{\alpha} Q' \quad \text{bn}(\alpha) \cap \text{fn}(P) = \emptyset}{P \mid Q \xrightarrow{\alpha} P \mid Q'}$$

Definition

$$\frac{P \xrightarrow{\bar{x}y} P' \quad Q \xrightarrow{x(y)} Q'}{P \mid Q \xrightarrow{\tau} P' \mid Q'} \quad \text{S-COM-L}$$

$$\frac{P \xrightarrow{x(y)} P' \quad Q \xrightarrow{\bar{x}y} Q'}{P \mid Q \xrightarrow{\tau} P' \mid Q'} \quad \text{S-COM-R}$$

$$\frac{P \mid !P \xrightarrow{\alpha} P'}{!P \xrightarrow{\alpha} P'} \quad \text{S-REPL}$$

$$\frac{P \xrightarrow{\alpha} P' \quad z \notin n(\alpha)}{(\nu z)P \xrightarrow{\alpha} (\nu z)P'} \quad \text{S-RES}$$

$$\frac{P \xrightarrow{\bar{x}(z)} P' \quad Q \xrightarrow{x(z)} Q' \quad z \notin \text{fn}(Q)}{P \mid Q \xrightarrow{\tau} (\nu z)(P' \mid Q')} \quad \text{S-CLOSE-L}$$

$$\frac{P \xrightarrow{\bar{x}z} P' \quad z \neq x}{(\nu z)P \xrightarrow{\bar{x}(z)} P'} \quad \text{S-OPEN}$$

$$\frac{P \xrightarrow{x(z)} P' \quad Q \xrightarrow{\bar{x}(z)} Q' \quad z \notin \text{fn}(P)}{P \mid Q \xrightarrow{\tau} (\nu z)(P' \mid Q')} \quad \text{S-CLOSE-R}$$

Examples of transitions

Suppose a server S controls access to a printer P and a client C wish to use it.

Access to the printer is represented by a name z , which is originally *local* to P and S ; the server S can send this private link to other processes alongside some channel y .

Examples of transitions

Suppose a server S controls access to a printer P and a client C wish to use it.

Access to the printer is represented by a name z , which is originally *local* to P and S ; the server S can send this private link to other processes alongside some channel y .

The system {server, printer} can make the following transition through a bound output action:

$$\text{S-OPEN} \frac{\bar{y}z.S \mid P \xrightarrow{\bar{y}z} S \mid P \quad y \neq z}{(\nu z)(\bar{y}z.S \mid P) \xrightarrow{\bar{y}(z)} S \mid P}$$

Examples of transitions

The client C can receive the name z through the same channel y ; we can represent this through the following transition:

$$\text{S-IN} \frac{}{y(x).C \xrightarrow{\bar{y}(z)} C\{z/x\}}$$

Examples of transitions

If we consider the system $\{\text{server}, \text{printer}, \text{client}\}$ as a whole, we can say that the client received the private link z from the server.

The link z , which used to be local to S and P , is now local to the client as well: the scope of z has changed.

This phenomenon is called **scope extension**.

Examples of transitions

If we consider the system $\{\text{server}, \text{printer}, \text{client}\}$ as a whole, we can say that the client received the private link z from the server.

The link z , which used to be local to S and P , is now local to the client as well: the scope of z has changed.

This phenomenon is called **scope extension**.

We can represent this with the following transition:

$$\text{S-CLOSE-L} \quad \frac{(\nu z)(\bar{y}z.S \mid P) \xrightarrow{\bar{y}(z)} S \mid P \quad y(x).C \xrightarrow{\bar{y}z} C\{z/x\}}{((\nu z)(\bar{y}z.S \mid P)) \mid y(x).C \xrightarrow{\tau} (\nu z)((S \mid P) \mid C\{z/x\})}$$

In reduction semantics, congruence relates syntactically equivalent processes; then processes are reduced up to syntactic equivalence.

In reduction semantics, congruence relates syntactically equivalent processes; then processes are reduced up to syntactic equivalence.

Definition

We define the **congruence** relation $\equiv$ as the smallest relation over $Proc$, closed under compatibility and equivalence laws, satisfying the following axioms:

SUM-ASSOC

$$\overline{P + (Q + R) \equiv (P + Q) + R}$$

SUM-UNIT

$$\overline{P + 0 \equiv P}$$

SUM-COMM

$$\overline{P + Q \equiv Q + P}$$

PAR-ASSOC

$$\overline{P \mid (Q \mid R) \equiv (P \mid Q) \mid R}$$

PAR-UNIT

$$\overline{P \mid 0 \equiv P}$$

PAR-COMM

$$\overline{P \mid Q \equiv Q \mid P}$$

Definition

SC-EXT-ZERO

$$\frac{}{(\nu x) 0 \equiv 0}$$

SC-EXT-SUM

$x \notin \text{fn}(Q)$

$$\frac{}{(\nu x)P + Q \equiv (\nu x)(P + Q)}$$

SC-EXT-PAR

$x \notin \text{fn}(Q)$

$$\frac{}{(\nu x)P \mid Q \equiv (\nu x)(P \mid Q)}$$

SC-EXT-RES

$$\frac{}{(\nu x)(\nu y)P \equiv (\nu y)(\nu x)P}$$

Definition

We define the **reduction** relation $\rightarrow$ as the smallest relation over $Proc$ satisfying the following rules:

$$\begin{array}{c} \text{R-COM} \\ \hline (R + \bar{x}y.P) \mid (S + x(z).Q) \rightarrow P \mid Q\{y/z\} \end{array} \qquad \begin{array}{c} \text{R-PAR} \\ P \rightarrow Q \\ \hline P \mid R \rightarrow Q \mid R \end{array}$$
$$\begin{array}{c} \text{R-RES} \\ P \rightarrow Q \\ \hline (\nu x)P \rightarrow (\nu x)Q \end{array} \qquad \begin{array}{c} \text{R-STRUCT} \\ P \equiv P' \quad P' \rightarrow Q' \quad Q' \equiv Q \\ \hline P \rightarrow Q \end{array}$$

Examples of congruences and reductions

Let's consider the same system $\{\text{server}, \text{printer}, \text{client}\}$ and describe its behaviour with the reduction semantics.

First, we can work on the process representing the system $\{\text{server}, \text{printer}, \text{client}\}$. Since the name z doesn't occur free in $y(x).C$, we have the following congruence:

$$((\nu z)(\bar{y}z.S \mid P)) \mid y(x).C \equiv (\nu z)((\bar{y}z.S \mid P) \mid y(x).C) \quad (3)$$

Examples of congruences and reductions

Let's consider the same system {server, printer, client} and describe its behaviour with the reduction semantics.

First, we can work on the process representing the system {server, printer, client}. Since the name z doesn't occur free in $y(x).C$, we have the following congruence:

$$((\nu z)(\bar{y}z.S \mid P)) \mid y(x).C \equiv (\nu z)((\bar{y}z.S \mid P) \mid y(x).C) \quad (3)$$

The last process can make the following reduction through a R-COM rule, followed by a R-RES rule:

$$\overline{(\nu z)((\bar{y}z.S \mid P) \mid y(x).C)} \rightarrow (\nu z)((S \mid P) \mid C\{z/x\})$$

Examples of congruences and reductions

The process $(\nu z)((S \mid P) \mid C\{z/x\})$ is syntactically equivalent to itself by reflexivity (4).

As a consequence, by applying a R-STRUCT rule we obtain the following reduction:

$$\frac{(3) \quad (\nu z)((\bar{y}z.S \mid P) \mid y(x).C) \rightarrow (\nu z)((S \mid P) \mid C\{z/x\}) \quad (4)}{((\nu z)(\bar{y}z.S \mid P)) \mid y(x).C \rightarrow (\nu z)((S \mid P) \mid C\{z/x\})}$$

It can be proved that **the two semantics are equivalent**, in the sense given by the following theorems:

Theorem (Theorem 1)

$P \xrightarrow{\tau} Q$ implies $P \rightarrow Q$.

It can be proved that **the two semantics are equivalent**, in the sense given by the following theorems:

Theorem (Theorem 1)

$P \xrightarrow{\tau} Q$ implies $P \rightarrow Q$.

Theorem (Theorem 2)

$P \rightarrow Q$ implies the existence of a Q' such that $P \xrightarrow{\tau} Q'$ and $Q \equiv Q'$.

As in the CCS, behavioural equivalence can be given by a bisimulation relation.

There are difference notions of bisimulation according to different definitions of observations.

My thesis work consists in the formalization of the aforementioned definitions and theorems through **Beluga**.

My thesis work consists in the formalization of the aforementioned definitions and theorems through **Beluga**.

Beluga is a functional programming language designed for reasoning about formal systems. It has been developed at the Complogic group at McGill University, led by Professor Brigitte Pientka.

In Beluga, binding constructs are encoded with a *higher order abstract syntax* (HOAS). Beluga also supports *contexts* and *contextual objects*, which can be used for hypothetical and parametric derivations. Proofs in Beluga are represented by *recursive programs*, according to the Curry-Howard isomorphism.

More specifically, after formalizing CCS definitions, I worked on the formalization of π -calculus syntax and semantics, following Honsell's approach to implement labelled transition semantics.

Currently I'm working on the formalization of the aforementioned theorems regarding the equivalence of the two semantics for π -calculus.

This work contributes to the *Concurrent Calculi Formalization Benchmark*, a set of benchmarks intended to stimulate the research and development of techniques used to formalize process calculi.

Thank You